

Módulo 2: Manipulación y Análisis de Datos

Introducción

Los datos rara vez llegan listos para analizar. Primero hay que inspeccionarlos, seleccionar lo importante, corregir valores faltantes y resumir patrones.

En este módulo usarás NumPy para trabajar con arreglos numéricos y Pandas para organizar datos tabulares.

Reto del módulo

Pregunta guía: ¿Qué estaciones ambientales presentan mediciones anómalas y cómo podemos justificarlo con datos?

Producto final: un resumen por estación con cantidad de mediciones, promedio de temperatura y conteo de valores fuera de rango.

Criterios de éxito:

- inspecciona la estructura del conjunto de datos;
- selecciona columnas y filtra filas con una condición;
- trata valores faltantes sin ocultar la decisión tomada;
- agrupa por estación;
- calcula métricas reproducibles;
- valida el resultado con pruebas pequeñas.

1. Mapa del proceso

```

Pregunta
↓
Cargar o construir datos
↓
Inspeccionar columnas, tipos y faltantes
↓
Limpiar y transformar
↓
Filtrar mediciones relevantes
↓
Agrupar por estación
↓
Calcular resumen y validar

```

Pseudocódigo

```

crear tabla de mediciones
revisar forma, columnas y valores faltantes
rellenar o excluir faltantes según una regla documentada
marcar temperaturas fuera del rango esperado
agrupar por estación
calcular cantidad, promedio y anomalías
comprobar el resultado con asserts

```

2. NumPy: arreglos numéricos

NumPy representa colecciones numéricas con `ndarray`. Sus operaciones vectorizadas permiten aplicar una operación a muchos valores sin escribir un ciclo explícito.

```
import numpy as np

temperaturas = np.array([18.5, 21.0, 19.2, 25.4])

promedio = temperaturas.mean()
maxima = temperaturas.max()
anomalias = temperaturas[temperaturas > 24]
```

Propiedades útiles

Propiedad o método	Uso
<code>array.shape</code>	dimensiones
<code>array.ndim</code>	número de dimensiones
<code>array.dtype</code>	tipo numérico
<code>array.mean()</code>	promedio
<code>array.sum()</code>	suma
<code>array.min() / max()</code>	extremos
<code>array[condicion]</code>	filtrado booleano

Operaciones vectorizadas

```
conversion = temperaturas * 1.8 + 32
es_calida = temperaturas >= 24
```

La expresión `temperaturas >= 24` produce un arreglo booleano. Ese arreglo puede seleccionar solo las filas o valores que cumplen una condición.

3. Pandas: datos tabulares

Un DataFrame organiza datos en filas y columnas, como una tabla.

```
import pandas as pd

datos = pd.DataFrame({
    "estacion": ["Norte", "Sur", "Norte"],
    "temperatura": [18.5, 25.4, 19.2],
})
```

Una columna individual es una Series:

```
datos["temperatura"]
```

Inspección inicial

```
datos.head()
datos.shape
datos.columns
datos.info()
datos.describe(numeric_only=True)
datos.isna().sum()
```

Antes de transformar datos, responde:

1. ¿Cuántas filas y columnas hay?
2. ¿Qué representa cada fila?
3. ¿Qué tipos tienen las columnas?
4. ¿Hay valores faltantes o duplicados?

4. Selección y filtrado

Seleccionar columnas

```
seleccion = datos[["estacion", "temperatura"]]
```

Filtrar filas

```
calidas = datos[datos["temperatura"] >= 24]  
norte = datos[datos["estacion"] == "Norte"]
```

Para combinar condiciones usa & y |, encerrando cada condición entre paréntesis:

```
norte_calido = datos[  
    (datos["estacion"] == "Norte") &  
    (datos["temperatura"] >= 24)  
]
```

Error frecuente: usar and o or con una columna completa. Pandas necesita & y | para comparar elemento por elemento.

5. Crear columnas y transformar datos

```
datos["temperatura_f"] = datos["temperatura"] * 1.8 + 32
datos["fuera_de_rango"] = (
    (datos["temperatura"] < 10) |
    (datos["temperatura"] > 24)
)
```

Una columna derivada debe tener un nombre que explique la regla aplicada. Documenta el rango usado: en este módulo se considerará anómala una temperatura menor que 10 o mayor que 24 grados Celsius.

6. Valores faltantes

Un valor faltante no significa automáticamente cero.

```
faltantes = datos.isna().sum()
datos_sin_faltantes = datos.dropna(subset=["temperatura"])
datos_rellenos = datos.copy()
datos_rellenos["humedad"] = datos_rellenos["humedad"].fillna(
    datos_rellenos["humedad"].median()
)
```

Elige la estrategia según el contexto:

Estrategia	Cuándo usarla
dropna	faltan pocas filas y no son críticas
fillna con mediana	variable numérica con valores extremos
valor explícito	la ausencia tiene significado conocido
conservar y marcar	el faltante también es información

Siempre registra qué hiciste y cuántos valores afectaste.

7. Agrupación y resumen

groupby divide los datos en grupos y permite aplicar una función a cada grupo.

```
resumen = (  
    datos.groupby("estacion")  
    .agg(  
        mediciones=("temperatura", "count"),  
        temperatura_promedio=("temperatura", "mean"),  
        maximo=("temperatura", "max"),  
    )  
    .reset_index()  
)
```

Para contar anomalías, crea primero una columna booleana:

```
datos["fuera_de_rango"] = (  
    (datos["temperatura"] < 10) |  
    (datos["temperatura"] > 24)  
)  
  
resumen["anomalías"] = datos.groupby("estacion")["fuera_de_rango"].sum().values
```

En una solución robusta, calcula todas las métricas dentro del mismo agg para conservar la correspondencia entre estaciones.

8. Validación

Los resultados deben poder comprobarse.

```
assert set(resumen["estacion"]) == {"Norte", "Sur", "Centro"}
assert resumen["mediciones"].sum() == len(datos)
assert (resumen["temperatura_promedio"] > 0).all()
```

Una prueba útil verifica una propiedad, no solo una salida concreta. Por ejemplo, el número total de mediciones agrupadas debe coincidir con el número de filas válidas de la tabla original.

9. Actividad del estudiante

Abre `01_manipulacion_y_analisis.ipynb` para seguir el ejemplo guiado y luego completa `02_reto_manipulacion.py`.

Reto 1: reproducción

Construye el resumen por estación usando las columnas indicadas.

Reto 2: adaptación

Añade el promedio de humedad y ordena el resultado de mayor a menor temperatura promedio.

Reto 3: extensión

Calcula la proporción de mediciones anómalas por estación y clasifica cada estación como estable o revisar.

Evidencia y evaluación

Criterio	Logrado cuando...
Inspección	identificas forma, tipos y faltantes
Limpieza	documentas qué haces con los valores faltantes
Selección	filtras con condiciones correctas
Agrupación	produces un resumen por estación
Interpretación	explicas qué estación requiere revisión
Validación	incluyes asserts o comprobaciones equivalentes

Reflexión final

1. ¿Qué diferencia hay entre una Series y un DataFrame?
2. ¿Por qué un valor faltante no debe reemplazarse automáticamente por cero?
3. ¿Qué métrica adicional pedirías antes de recomendar una acción sobre una estación?